Token-Pruned Dynamic ViT with LoRA Adaptation for Efficient On-Device Inference

FRANÇOIS Gabriel
ENSAE

LEROY Léo
ENSAE

EL BAZ Avner
ENSAE

Abstract

Vision Transformers (ViTs) achieve state-of-the-art performance in numerous computer vision tasks, but their computational and memory footprint remains prohibitive for resource-constrained devices such as smartphones, drones, and embedded sensors. In this work, we tackled this problem by developing a pipeline that combines dynamic token pruning (DynamicViT) with low-rank adaptation (LoRA). This enabled to reduce both inference-time operations and fine-tuning costs. DynamicViT progressively removes non-informative tokens conditioned on the input, while LoRA injects lightweight trainable parameters to adapt large pre-trained ViTs without modifying their full weight matrices. Our worked confirmed the reproducibility of Dynamic ViT. To make training more stable, we added an adaptive token pruning schedule. Our code is available here.

1 Introduction

Deep learning vision models have achieved remarkable progress in classification, detection, and segmentation. However, models become increasingly heavier. Their size create such energy consumptions that large private companies can handle them. This creates problems of privacy and latency while accessing remote servers as well as an environmental danger.

The primary computational bottleneck of Vision Transformers (ViTs) lies in the self-attention mechanism. ViTs decompose an image into a sequence of smaller images called "patches", each embedded as a token, and process this sequence through stacked self-attention layers. In each layer, self-attention computes pairwise interactions between all tokens, allowing the model to capture long-range dependencies and global context. For a sequence of N tokens, self-attention has $O(N^2)$ time and memory complexity. As image resolution increases, the number of tokens grows rapidly, making ViTs computationally very expensive.

Prior works (Wang et al. 2020) on efficient transformers mainly relies on approximating or sparsifying self-attention through fixed architectural modifications or linearized attention mechanisms, which remain input-agnostic. These approaches still process all tokens at every layer, leading to poor scalability with image resolution.

Empirical analyses of attention maps reveal a significant redundancy in token contributions: although attention is computed over all token pairs, only a subset of tokens meaningfully influences the final prediction. DynamicViT, proposed by Rao, Zhao, et al. 2021, addresses this inefficiency by dynamically estimating token saliency during inference and progressively pruning tokens deemed non-essential. By dynamically reducing the token set as depth increases, DynamicViT preserves predictive performance while substantially lowering computational cost, without requiring a fixed sparsity pattern.

While dynamic token pruning reduces inference-time complexity, it does not address the separate challenge of efficiently fine-tuning large ViTs. Full fine-tuning remains computationally and memory intensive due to the sheer number of parameters. To alleviate this, we incorporate Low-Rank

Final year project - Advanced Machine Learning (ENSAE 2026), Palaiseau, France.

Adaptation (LoRA), which freezes the original model weights and introduces small, trainable low-rank matrices into selected layers. This approach significantly reduces the number of trainable parameters and training cost, while retaining the expressive capacity of the pretrained model. Combined with DynamicViT, LoRA enables both efficient inference and parameter-efficient adaptation of large-scale Vision Transformers.

The experiments in this work rely on two image classification datasets: ImageNet-1k and STL-10. We also used CIFAR-100 to test our first implementations. It contained 100 classes containing 600 images each of dimension 32x32 in color.

ImageNet-1k contains 1.28 million training images and 50,000 validation images spanning 1,000 categories. In our experiments, all ImageNet images are resized to a resolution of 128×128 prior to training and evaluation.

STL-10 is a 10-class image classification dataset consisting of color images of size 96×96. The labeled dataset contains 500 training images per class (organized into 10 predefined folds) and 800 test images per class, for a total of 5,000 training images and 8,000 test images. In addition, STL-10 provides 100,000 unlabeled images extracted from a broader distribution of ImageNet images; these unlabeled samples are not used in our experiments.

In this paper, we present a unified framework combining DynamicViT and LoRA to produce an efficient, deployable model suitable for edge devices.

2 Dynamic ViT

2.1 Vision Transformer (ViT)

The Vision Transformer (ViT) adapts the standard Transformer architecture for image recognition by treating an image as a sequence of patches Dosovitskiy et al. 2021. Given an input image $x \in \mathbb{R}^{H \times W \times C}$, ViT splits it into N fixed-size non-overlapping patches, where $N = HW/P^2$ is the sequence length and P is the patch size.

These patches are linearly projected onto an embedding space of dimension D and summed with learnable position embeddings to retain spatial information. A special learnable `[CLS]` token is prepended to the sequence to serve as the aggregate image representation for classification.

The sequence of embeddings z_0 is processed by L Transformer encoder layers. Each layer consists of Multi-Head Self-Attention (MSA) and a Feed-Forward Network (FFN), with Layer Normalization (LN) and residual connections:

$$z'_l = \text{MSA}(\text{LN}(z_{l-1})) + z_{l-1}, \tag{1}$$

$$z_l = \text{FFN}(\text{LN}(z'_l)) + z'_l, \tag{2}$$

where the core Self-Attention operation for input matrices Q, K, V is defined as:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V. \tag{3}$$

The final classification is performed by a linear head attached to the output state of the `[CLS]` token.

2.2 The core concept: dynamic token sparsification

The core idea behind Dynamic ViT, introduced by Rao, Zhao, et al. 2021, is to leverage the spatial sparsity inherent in image data to accelerate Vision Transformers. In standard Vision Transformers, all patch embeddings are processed uniformly across every layer, even though the final prediction often depends on only a small subset of highly informative tokens. Dynamic ViT addresses this inefficiency by selectively focusing computation on the most relevant tokens while discarding less useful ones.

To better understand the intuition behind this model, we can take the example of an automated scheduling algorithm designed to predict the optimal time for a door to close. Although the system initially considers many input variables, it ultimately discovers that the prediction depends almost entirely on a single factor: the hour of the day. This turned the remaining inputs unnecessary. Dynamic

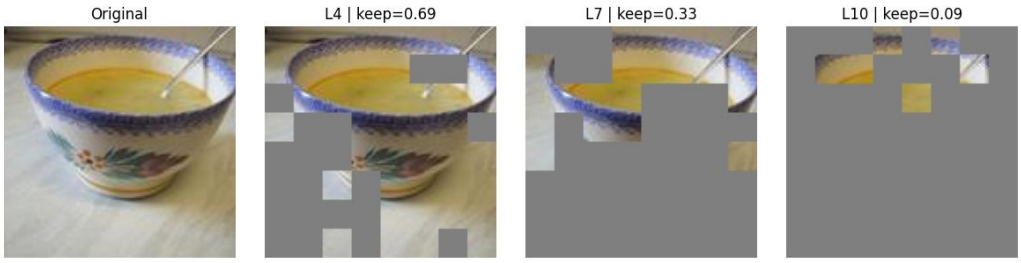

Figure 1: Visualization of dynamic token pruning on a coffee cup image from ImageNet, showing that the most relevant parts of the picture are the edges to determine it is a cup.

ViT dynamically adapts the token selection, as the relevant ones will depend on the orientation of the objects and other configurations (see Fig. 1).

To performs this dynamic selection, Dynamic ViT introduces a hierarchical pruning framework. We insert prediction modules at specific layers of the network (e.g., before the 4th, 7th, and 10th layers). These modules dynamically decide which patch tokens are preserved and which are discarded.

We maintain a binary decision mask $D = \{0, 1\}^N$, where N is the number of patch embeddings (flattened $H \times W$) This mask is initialized to 1 for all tokens. As the computation proceeds through the network, the mask is updated progressively using the rule $\hat{D} \leftarrow \hat{D} \odot D$. This ensures that once a token is dropped (set to 0), it is never used again in subsequent layers.

2.3 The prediction module: deciding what to keep

The decision to keep or drop a token is made by a lightweight prediction module that analyzes both the token itself and the global image context. The module takes the current tokens x and the current mask $\hat{D}$ as inputs and computes two feature vectors:

- **Local Features** (z^{local}): A Multi-Layer Perceptron (MLP) projects the individual features of each token ($C \rightarrow C$)
- **Global Features** (z^{global}): An aggregation function (average pooling) computes a context vector from the remaining tokens only (where $\hat{D}_i = 1$).

These local and global embeddings are concatenated and passed through a final MLP to predict the probability π of keeping or dropping each token:

$$\pi = \text{Softmax}(\text{MLP}([z^{\text{local}}, z^{\text{global}}]))$$

2.4 End-to-end optimization: training challenges

Training this binary decision framework presents two major hurdles: differentiability and hardware parallelization.

Differentiability via Gumbel-softmax: Sampling a binary mask D directly from probabilities is non-differentiable (cf. Rao, Zhao, et al. 2021), which breaks backpropagation. To solve this, the framework utilizes Gumbel-Softmax sampling. This technique allows the model to sample the decision mask D while allowing gradients to flow back into the prediction module during end-to-end training.

Parallelization via attention masking: Physically discarding tokens during training is problematic because it results in irregular tensor shapes (ragged batches) that prevents efficient GPU parallelization. Instead of removing tokens, we use Attention Masking to "soft-drop" them. We compute a graph $G_{i,j}$ where connections to dropped tokens are severed, but the matrix shape $N \times N$ remains constant. The modified attention calculation becomes:

$$A_{i,j} = \frac{\exp(P_{i,j})G_{i,j}}{\sum_{k=1}^{N} \exp(P_{i,k})G_{i,k}}$$

Here, if a token is dropped (*e.g.* : $\hat{D} = 0$), it contributes nothing to the updates of other tokens, effectively neutralizing it without breaking the batch structure required for GPU training.

2.5 Training Objectives and Strategy

The training process uses the original, full-token Vision Transformer as a "teacher" model to guide the sparse "student" (Dynamic ViT). The total loss function combines four components:

- **Classification loss** ($\mathcal{L}_{\text{cls}}$): Standard cross-entropy loss on the final prediction.

  $$L_{\text{cls}} = -\sum_{i=1}^{C} y_i \log(p_i)$$

  where y_i is the true label and p_i is the predicted probability for class i and C is the number of labels.

- **Distillation loss** ($\mathcal{L}_{\text{distill}}$): An MSE loss that forces the student's remaining token features to match the teacher's features.

  $$L_{\text{distill}} = \frac{1}{\sum_{b=1}^{B}\sum_{i=1}^{N} \hat{D}_i^{b,S}} \sum_{b=1}^{B}\sum_{i=1}^{N} \hat{D}_i^{b,S} \left(t_i - t_i'\right)^2$$

  where:

  - B is the batch size,
  - N is the number of tokens,
  - $\hat{D}_i^{b,S}$ is the importance score of the i-th token in the b-th sample at stage S,
  - t_i and t_i' are the token representations of the student and teacher models, respectively.

- **KL divergence** ($\mathcal{L}_{\text{KL}}$): Minimizes the difference between the student's and teacher's final prediction distributions.

  $$L_{\text{KL}} = D_{\text{KL}}(P_t \| P_s) = \sum_{i=1}^{C} P_t(i) \log\left(\frac{P_t(i)}{P_s(i)}\right)$$

  where P_t and P_s are the teacher's and student's probability distributions over C labels.

- **Ratio loss** ($\mathcal{L}_{\text{ratio}}$): An MSE constraint that ensures the model maintains the target keeping ratio ρ (e.g., keeping 70% of tokens per stage).

  $$L_{\text{ratio}} = \frac{1}{BS} \sum_{b=1}^{B}\sum_{s=1}^{S} \left(\rho^{(s)} - \frac{1}{N}\sum_{i=1}^{N} \hat{D}_i^{b,s}\right)^2$$

  where:

  - S is the number of stages,
  - $\rho^{(s)}$ is the target keeping ratio for stage s,
  - $\hat{D}_i^{b,s}$ is the predicted importance score of the i-th token in the b-th sample at stage s.

Training Strategy ("Warm-up"): To ensure stability, the backbone weights are fixed for the first 5 epochs. This allows the prediction modules to learn meaningful masking policies before the backbone begins adapting to the sparse inputs.

3 Details on Dynamic ViT implementation

3.1 Model specification and implementation choices

In this work, we adapt the Dynamic Vision Transformer (Dynamic ViT) framework to a constrained experimental setting, requiring a substantial reduction of model capacity compared to the configurations commonly used in the literature for ImageNet-scale experiments. All experiments are conducted on a single NVIDIA T4 GPU with 16 GB of memory, which imposes strict computational and memory constraints and motivates several architectural and training design choices.

The reference works on Dynamic ViT rely on models derived from DeiT-S, which constitutes the smallest architecture for which results are reported in the DynamicViT research paper Rao, Zhao, et al. 2021. This model is characterized by 12 transformer layers, 6 attention heads, an embedding dimension of 384, and operates on high-resolution inputs of 384×384 pixels with a patch size of 16×16. Depending on the training resolution, this results in approximately 192 to 576 tokens per image.

In contrast, our implementation preserves the overall transformer depth (12 layers) but significantly reduces the model width and token count. Specifically, we use 4 attention heads, an embedding dimension of 96, and input images resized to 128×128 pixels, while keeping the same patch size. This configuration yields only 64 tokens per image, leading to a substantially lower representational capacity and reduced redundancy across tokens.

These architectural differences are critical when interpreting the performance of our Dynamic ViT model. The aggressive reduction in both embedding dimension and token count limits the expressive power of the network and constrains the effectiveness of token pruning strategies, which in the original Dynamic ViT setting benefit from a large pool of redundant tokens.

3.2 Adaptive token pruning strategy

To compensate for the reduced model capacity and stabilize training, we adopt an adaptive pruning schedule rather than a fixed pruning ratio. Token pruning is applied at layers $\{4, 7, 10\}$, following the design choice of the original Dynamic ViT framework, where pruning is introduced progressively at intermediate and late transformer blocks. This placement avoids early pruning of low-level visual features while allowing higher-level semantic tokens to be selectively discarded.

The token retention ratio ρ is gradually adjusted during training. Specifically, ρ follows a sigmoid schedule decreasing smoothly from $\rho_{\text{init}} = 1$ to $\rho_{\text{final}} = 0.7$ over 100 training epochs, with a sigmoid steepness parameter set to 10.0. This value was chosen empirically to ensure a sufficiently sharp transition between the unpruned and pruned regimes, while avoiding abrupt changes that were observed to destabilize optimization in preliminary experiments.

During the early stages of training, pruning is effectively disabled, allowing the model to learn robust low-level and mid-level representations without information loss. We empirically observed that applying strong pruning from the first epochs leads to an immediate collapse of performance, with classification accuracy dropping to approximately 5%.

As training progresses, pruning is progressively increased, encouraging the model to identify and preserve informative tokens while discarding redundant ones. In the final epochs, the sigmoid schedule flattens, reducing the rate of pruning changes and ensuring training stability. This gradual strategy differs from the original Dynamic ViT setting, where larger models and higher token redundancy allow for more aggressive pruning schedules.

The training objective combines several loss terms weighted by fixed coefficients. The classification loss is scaled by $\lambda_{\text{class}} = 0.001$, ensuring that it remains numerically stable when combined with auxiliary losses. The distillation objective relies on a Kullback-Leibler divergence weighted by $\lambda_{\text{KL}} = 1$, which enforces strong alignment between the student and teacher predictions. A token ratio regularization term weighted by $\lambda_{\text{ratio}} = 0.5$ encourages adherence to the target retention ratio throughout training. Finally, the feature-level distillation loss is scaled by $\lambda_{\text{distill}} = 10^{-4}$, providing a weak but stabilizing supervision signal without dominating the optimization. These weights were selected empirically to balance stability and convergence in a low-capacity regime.

3.3 Training efficiency

To mitigate the computational overhead induced by dynamic routing and pruning decisions, we employ mixed-precision training using automatic casting (`autocast`), together with several minor implementation-level optimizations. These choices significantly reduce both memory consumption and training time, which is critical given the limited hardware budget.

Overall, these optimizations yield a speed-up of approximately $\times 1.6$ compared to full-precision training. In practice, the average training time is reduced from roughly 50 hours to approximately 30 hours for 100 epochs on ImageNet-1k when training on a single NVIDIA T4 GPU with 16 GB of memory.

This acceleration makes it feasible to train the Dynamic ViT student model end-to-end within the available computational constraints. Due to the high computational cost, all reported ImageNet results correspond to a single training run. Overall, our implementation constitutes a constrained yet realistic adaptation of Dynamic ViT, designed to study adaptive token pruning under significantly reduced architectural and computational budgets.

4 Dynamic Vision Transformer: experimental results

The performance gap observed between our Dynamic ViT model and the results reported in the original Dynamic ViT work on ImageNet is primarily explained by the large discrepancy in backbone capacity and input resolution, rather than by limitations of the adaptive token pruning mechanism itself. In contrast to the original setting, which builds upon a strong DeiT-S backbone pretrained and fine-tuned at high resolution, our model operates in a severely constrained regime, with a reduced embedding dimension, fewer attention heads, and a significantly smaller number of input tokens.

Table 1 summarizes the main architectural differences between the original Dynamic ViT configuration and our implementation, together with the corresponding top-1 accuracy obtained on ImageNet-1k. It is important to note that, in our setting, only the pruned Dynamic ViT model is evaluated, as pruning is enabled throughout training following an adaptive schedule.

In the original Dynamic ViT configuration, pruning removes a large fraction of tokens while inducing only a marginal drop in accuracy, reflecting the high degree of token redundancy present in the DeiT-S backbone. In our case, the substantially lower accuracy reflects the limited capacity of the student model and the reduced input resolution. Nevertheless, the adaptive pruning strategy allows stable training and maintains performance despite operating close to the capacity limit, where little redundancy is available for sparsification.

From this perspective, our results should not be interpreted as an attempt to match the performance of state-of-the-art Dynamic ViT models, but rather as evidence that adaptive token pruning remains viable and stable even in low-capacity regimes with a limited token budget.

	Dynamic ViT (DeiT-S)	**Our Dynamic ViT model**
Backbone depth (layers)	12	12
Embedding dimension	384	96
Attention heads	6	4
Input resolution	384^2	128^2
Tokens per image	192–576	64
Retention ratio ρ	0.7	≈ 0.7
Top-1 accuracy (ImageNet-1k)	81.4%	37.4% (student) / 42.2% (teacher)
Parameters	23 M	1.56 M
FLOPs per image	15.5 GFLOPs	0.12GFLOPs

Table 1: Comparison between the original Dynamic ViT setup and our constrained implementation on ImageNet-1k.

Beyond final accuracy, adaptive pruning was found to have a strong impact on training stability and convergence of our student model. In preliminary experiments without adaptive scheduling, training consistently collapsed, with ImageNet top-1 accuracy remaining in the 3–10% range. By contrast, enabling adaptive pruning allowed the model to converge reliably, reaching a final top-1 accuracy of 37.4% on ImageNet-1k.

Even under these constraints, the computational efficiency is remarkable. About 66% of tokens were pruned and it reduced computational FLOPs by around 33%:

- Teacher: 1.51 M parameters, 0.18 GFLOPs per image
- Student: 1.56 M parameters, 0.12 GFLOPs per image

For reference, the original Dynamic ViT-S backbone operates at 15.5 GFLOPs per image, with 22M parameters, achieving $\approx 81.6\%$ top-1 accuracy. Our models are therefore over 129 times smaller while still achieving meaningful results.

These results illustrate two key points:

1. Adaptive token pruning is robust, even in low capacity regimes.
2. Computationally lightweight models can still learn meaningful representations when pruning is combined with careful training schedules.

A similar effect was observed on a smaller-scale setting, specifically on the CIFAR-100 dataset, where the student model reached a test accuracy of 71.06% after only three training epochs with adaptive pruning, whereas non-adaptive pruning resulted in a much lower final accuracy of 57.56% even after full training on 100 epochs. These results suggest that adaptive pruning schedules play an important role in stabilizing optimization, particularly in low-capacity regimes.

Finally, combining token pruning with mixed-precision execution yields a measurable increase in effective throughput on GPU for our models trained on ImageNet. Inference speed improves from approximately 3063 images/s (teacher) to 3203 images/s (student), corresponding to a +4.55% speed-up. This confirms that dynamic token sparsification translates into tangible computational gains when sufficient parallelism is available.

In contrast, on CPU, the same pruning strategy yielded different results on the datasets. On CIFAR-100, the inference speed increased by 33%. For ImageNet, inference speed slightly increases from 3129 images/s (teacher) to 3206 images/s (student), corresponding to almost 2.5% gain in speed. The discrepancy might exist because, at a high throughput of over 3,000 images/s, the CPU's execution time is dominated by administrative overhead (indexing, sorting, and predictor logic) rather than raw matrix multiplication. Since ImageNet tensors are much larger and more complex to "slice" in memory, the time saved by processing fewer tokens might be almost entirely consumed by the physical cost of rearranging the data in the CPU's cache.

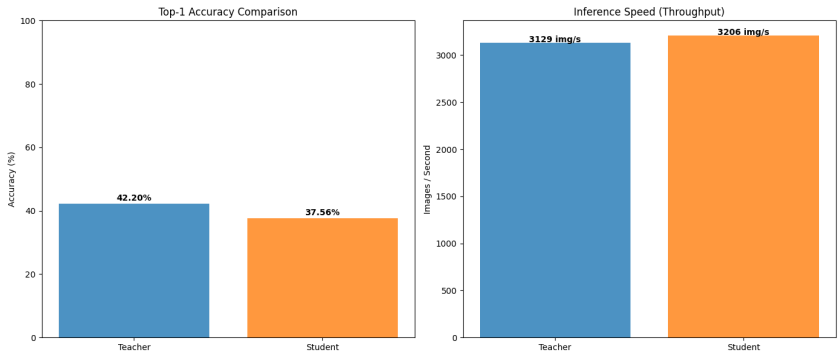

Figure 2: Performance comparison of our ViT and Dynamic ViT models on ImageNet (CPU)

5 Low-Rank Adaptation (LoRA)

5.1 Presentation

Fine-tuning large models is costly because modifying each parameter requires substantial memory and compute. LoRA (Hu et al., 2021) addresses this by decomposing parameter updates into low-rank matrices. The motivation behind this method is the empirical observation that, during finetuning, the

updates to pretrained weights often lie in a low dimensional subsapce. Rather than learning a full-rank update for each weight matrix, LoRA constrains the adaptation to be low rank. This significantly reduces the number of parameters.

Given a frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$, standard fine-tuning learns a full update matrix ΔW. LoRA instead parameterizes: $W = W_0 + BA$, where: W_0 is frozen weight matrix, $r \ll \min(d, k)$ is the low rank, and $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$ are trainable.

This formulation offers three advantages. It is memory efficient as trainable parameters scale with $r(d + k)$ instead of dk. Training is faster thanks to the low rank matrices. And finally, LoRA layers can be added or removed easily without altering the original architecture.

5.2 Experiment

We pretrained a student model on Imagenet dataset and then finetuned it on STL dataset. We applied LoRA to all linear layers.

Implementation details: The student uses a model dimension of $d_{\text{model}} = 96$ with 12 transformer layers and 4 attention heads. Images of size 128×128 with 3 channels are split into non-overlapping patches of size 16×16. The classifier is initially defined with 1000 classes to load the pretrained student model and is subsequently adapted to 10 classes for STL dataset. Input images are normalized using ImageNet statistics with mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. LoRA is applied with rank $r = 4$. It was finetuned for 100 epochs using a batch size of 256 and a learning rate $\alpha = 5 \times 10^{-3}$.

Results: While initially the performance was quite low, around 12% of accuracy, it very quickly reached more than 50% during training while this threshold took a lot of time to be reached when training student. The accuracy on the test dataset for a best finetuned model reached 65%. The training was really fast: only a few seconds per epoch. These results are strongly encouraging and show that LoRA can be applied seamlessly to DynamicViT model.

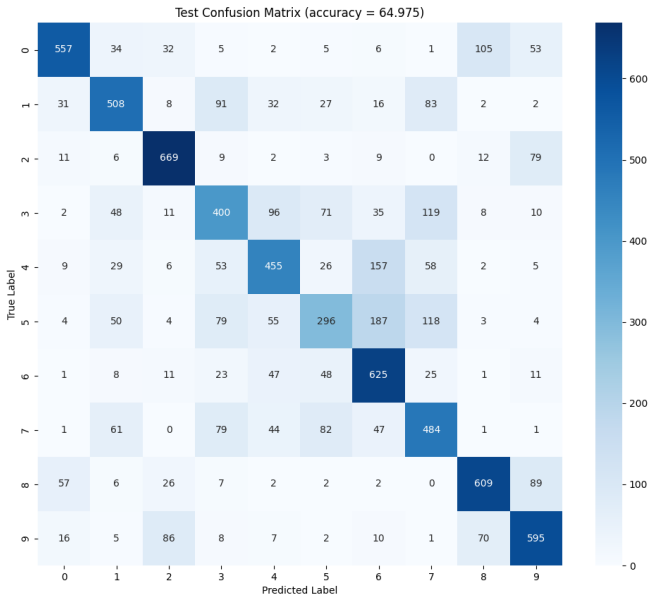

Figure 3: Confusion matrix for STL test dataset.

Conclusion

DynamicViT introduces data-dependent, unstructured sparsity into Vision Transformers. By hierarchically pruning up to 66% of input tokens, it reduces computational FLOPs by around 33% and improves inference throughput by over 4%, with negligible loss in accuracy. Even at a small model scale, this approach substantially lowers computational and memory costs, achieving 65% accuracy on STL-10 during fine-tuning.

The key for these results lie in the GPU-friendly dynamic sparsity. Unlike traditional CNN pruning, DynamicViT's unstructured sparsity modifies the underlying matrix operations, creating irregular memory access patterns. This makes it more CPU-friendly. In DynamicViT, masking during training and fixed-ratio pruning during inference partially mitigates these challenges on GPUs. In our implementation, we improved training stability thanks to an adaptive pruning schedule that gradually introduces sparsity, preventing performance collapse and enabling reliable convergence.

Despite its inference efficiency, the computational cost of training remains significant. The teacher–student framework requires training two models, limiting fully low-resource deployment. LoRA fine-tuning reduces trainable parameters and training time, enabling lightweight adaptation on smaller devices, but the need for an initial costly training phase remains a barrier.

The effectiveness of token pruning is also influenced by the teacher model and the attention mechanism. Using a compact Vision Transformer as the teacher limits the richness of the distillation signal. Teachers with stronger spatial inductive biases, such as convolutional or hybrid ConvNeXt-style architectures (Zhuang Liu et al. 2022; Rao, Zuyan Liu, et al. 2023), could provide more informative supervision and enhance pruning effectiveness. Similarly, the dense query-key self-attention employed here constrains practical efficiency gains. Attention mechanisms based on latent or learned queries, as in sparse models like BigBird Zaheer et al. 2020, offer a promising direction to further reduce computational costs. Integrating dynamic token pruning with such sparse or latent attention formulations represents a natural extension to improve end-to-end efficiency.

In summary, DynamicViT demonstrates that data-dependent token pruning with adaptive pruning schedule yields a stable and efficient training even in low-capacity Vision Transformers. Its combination with LoRA enables practical low-resource fine-tuning. Future work on richer teachers and sparse attention mechanisms promises further gains in both efficiency and applicability.

References

Dosovitskiy, Alexey et al. (2021). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *arXiv preprint arXiv:2010.11929*. DOI: 10.48550/arXiv.2010.11929. arXiv: 2010.11929 [cs.CV].

Liu, Zhuang et al. (2022). "A ConvNet for the 2020s". In: *CoRR* abs/2201.03545. arXiv: 2201.03545. URL: https://arxiv.org/abs/2201.03545.

Rao, Yongming, Zuyan Liu, et al. (2023). "Dynamic Spatial Sparsification for Efficient Vision Transformers and Convolutional Neural Networks". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.9, pp. 10883–10897. DOI: 10.1109/TPAMI.2023.3263826.

Rao, Yongming, Wenliang Zhao, et al. (2021). "DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification". In: *arXiv preprint arXiv:2106.02034*. DOI: 10.48550/arXiv.2106.02034. arXiv: 2106.02034 [cs.CV].

Wang, Sinong et al. (2020). "Linformer: Self-Attention with Linear Complexity". In: arXiv: 2006.04768 [cs.LG]. URL: https://arxiv.org/abs/2006.04768.

Zaheer, Manzil et al. (2020). "Big Bird: Transformers for Longer Sequences". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., pp. 17283–17297. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/c8512d142a2d849725f31a9a7a361ab9-Paper.pdf.